מרימי — תיקונים (Repairs)

קבלת תיקון, זרימת משימות ואינטגרציה עם QA

מפתחת 3 · Repairs Module

1. פתיחה — מה לומר (30 שניות)

> *"שלום, אני מרימי. אחראית על מודול התיקונים — מקבלת פאה לתיקון ועד מסירה ללקוחה. בניתי את טופס קבלת התיקון, את מנוע המשימות שמעביר אוטומטית לחפיפה ולבקרה, ואת מסך המשימות לעובדות."*

2. מה עשיתי

- **טופס קבלת תיקון (DiagnosisChecklist)** — זיהוי לקוחה, צילום פאה, בחירת סוגי תיקון.
- **קטגוריות תיקון מוגדרות (CATEGORY_MAP)** — צבע, מכונה, עבודת יד, חפיפה, בקרה.
- **שיבוץ עובדות לפי קטגוריה (StaffAllocator)** — עם הצגת עומס נוכחי.
- **מנוע זרימת משימות** — כשעובדת מסיימת, המערכת מעבירה אוטומטית לשלב הבא.
- **יצירה אוטומטית של חפיפה + בקרה** — בסיום כל התיקונים, גם אם לא נבחרו מראש.
- **מסך משימות לעובדות** — `repairs/tasks/` + שילוב בתחנת הייצור.
- **אינטגרציה עם QA** — יצירת משימת בקרה אוטומטית בסיום (מודול חני).

3. קבצים מרכזיים

שכבה	קובץ	תפקיד
Client	`components/Repairs/DiagnosisChecklist/DiagnosisChecklist.tsx`	טופס קבלת תיקון
Client	`components/Repairs/StaffAllocator/StaffAllocator.tsx`	שיבוץ עובדות לפי קטגוריה
Client	`components/Repairs/RepairWorkerList/RepairWorkerList.tsx`	רשימת משימות לעובדת
Client	`components/Repairs/TaskItem/TaskItem.tsx`	כרטיס משימה (משותף עם ייצור)
Server	`Models_Service/Repairs/repairModel.ts`	מודל + CATEGORY_MAP + Validation
Server	`Models_Service/Repairs/repairService.ts`	לוגיקת תיקונים + זרימה
Server	`Routers/repairRouter.ts`	API — `/api/repairs`
Tests	`tests/repair.test.ts`	בדיקות יצירה, משימות, מעבר שלב

4. הסבר על קטעי קוד

קטע 1 — קטגוריות תיקון (Server Model)

קובץ: repairModel.ts — CATEGORY_MAP

```
const CATEGORY_MAP = {
  'צבע': ['גוונים', 'שורש', 'שטיפה לעש', 'הבהרה לבלונד'],
  'מכונה': [
    'התקנת לייס', 'תיקון רשת', 'העברת רשת',
    'קיצור פאה', 'הארכת פאה', 'התקנת ריבן'
  ],
  'עבודת יד': ['מילוי לייס', 'בייביהיירי', 'גובה בקודקוד'],
  'חפיפה': ['חלק', 'מוברש', 'גלי', 'תלתלים', 'ייבוש טבעי'],
  'בקרה': ['בדיקה סופית']
};

// Validation – תת-קטגוריה חייבת להיות ברשימה של הקטגוריה
validate: {
  validator: function(value) {
    return CATEGORY_MAP[this.category]?.includes(value);
  }
}
```

מה קורה כאן:

- כל תיקון שייך ל**קטגוריה** (צבע, מכונה...) ו**תת-קטגוריה** ספציפית.
- ה-Validation ב-Mongoose מוודא שתת-הקטגוריה תקינה — מונע הזנת נתונים שגויים.
- המזכירה בוחרת מתוך רשימה קבועה, לא מקלידה טקסט חופשי.

קטע 2 — מנוע הזרימה האוטומטי (Server)

קובץ: repairService.ts — updateTaskAndMoveToNext

```
 //סימון משימה כהושלמה
currentTask.status = 'בוצע';
await repair.save();

// חיפוש המשימה הבאה
let nextTask = repair.tasks.find(t => t.status === 'ממתין');

// יצירת חפיפה ובקרה אוטומטית אם אין יותר משימות
if (!nextTask) {
  const hasWash = repair.tasks.some(t => t.category === 'חפיפה');
  const hasQA   = repair.tasks.some(t => t.category === 'בקרה');
  if (!hasWash || !hasQA) {
    repair.tasks.push({ category: 'חפיפה', subCategory: 'חלק', status: 'ממתין' });
    repair.tasks.push({ category: 'בקרה',  subCategory: 'בדיקה סופית', status: 'ממתין' });
    await repair.save();
```

```
    nextTask = repair.tasks.find(t => t.status === 'ממתין');
  }
}

if (nextTask.category === 'בקרה') {
  repair.overallStatus = 'בבקרה';
  await Service.create({
    serviceType: 'Repair QA', origin: 'Repair', status: 'QA'
  });
} else if (nextTask.category === 'חפיפה') {
  repair.overallStatus = 'בחפיפה';
}
```

מה קורה כאן:

- כשעובדת מסיימת משימה — הסטטוס משתנה ל-**"בוצע"**.
- אם אין משימות נוספות — המערכת **מוסיפה אוטומטית** חפיפה ובקרה.
- הסטטוס הכללי: בתיקון → בחפיפה → בבקרה.
- בהגעה לבקרה — נוצרת משימת QA (חני מטפלת באישור/פסילה).

קטע 3 — שליחת תיקון חדש (Client)

קובץ: DiagnosisChecklist.tsx — handleSubmit

```
const handleSubmit = async () => {
  if (!customerId || !wigCode || selectedTasks.length === 0) {
    alert("נא לוודא זיהוי לקוחה, קוד פאה ובחירת תיקונים");
    return;
  }

  const repairData = {
    customerId,
    wigCode,
    isUrgent,
    tasks: selectedTasks,       // מערך משימות עם קטגוריה + עובדת
    washerId,                   // עובדת חפיפה
    adminId,                    // בודקת QA
    images: capturedImages,     // צילומי הפאה
    overallStatus: 'בתיקון',
    internalNote: internalNote
  };

  const response = await axios.post('/repairs', repairData);
};
```

מה קורה כאן:

- Validation: חובה לקוחה, קוד פאה, ולפחות תיקון אחד.
- כל משימה כוללת: קטגוריה, תת-קטגוריה, עובדת משובצת, תאריך יעד.

- צילומי הפאה (לפני) נשמרים ב-`images / beforeImageUrl[]`.
- הנתונים נשלחים ל-`POST /api/repairs`.

5. תסריט הדגמה

1. התחברי כ-`admin` → עברי ל-`repairs/new/` (קבלת תיקון).
2. חפשי לקוחה לפי ת.ז. והזיני קוד פאה (למשל: `WIG-1234`).
3. צלמי את הפאה (עד 2 תמונות) — לפני ותקלה.
4. בחרי סוגי תיקון: למשל "גוונים" (צבע) + "תיקון רשת" (מכונה).
5. שבצי עובדת לכל קטגוריה + עובדת חפיפה + בודקת QA.
6. שלחי — הודעת הצלחה, התיקון נוצר ב-DB.
7. התחברי כעובדת (הודיה / `password123`) → `repairs/tasks/`.
8. לחצי "בוצע" — הסבירי שהמערכת מודיעה על התחנה הבאה.

6. טכנולוגיות

React · TypeScript · Axios · Mongoose Schema Validation · Express Router · MediaDevices API (צילום) · Jest + Supertest

7. שאלות ותשובות למורה

ש: איך נוצרת חפיפה אוטומטית?

ב-`updateTaskAndMoveToNext`, כשכל המשימות הידניות (צבע, מכונה, עבודת יד) הושלמו, הקוד בודק אם יש משימת חפיפה ומשימת בקרה. אם חסרות — הוא מוסיף אותן אוטומטית לרשימת המשימות. כך גם אם המזכירה שכחה לבחור חפיפה — הפאה לא "נתקעת" אלא ממשיכה בזרימה.

ש: מה ההבדל בין קטגוריה לתת-קטגוריה?

קטגוריה = תחום עבודה רחב (צבע, מכונה, עבודת יד, חפיפה, בקרה). **תת-קטגוריה** = סוג תיקון ספציפי בתוך הקטגוריה (למשל: "גוונים" תחת "צבע"). ה-`CATEGORY_MAP` מגדיר את הקשר ביניהם, וה-Validation מוודא שההתאמה נכונה.

ש: איך עובדת האינטגרציה עם QA (חני)?

כשהתיקון מגיע לשלב "בקרה", השרת יוצר רשומת `Service` עם `serviceType: 'Repair QA'` ו-`origin: 'Repair'`. חני רואה את זה בלוח QA שלה (`/qa`) ומאשרת/פוסלת. באישור — התיקון עובר ל-מוכן. בפסילה — המשימות חוזרות ל-ממתין עם `qaNote` ו-`qaRejectionPhoto`.

ש: מה קורה כשעובדת לוחצת "בוצע" פעמיים?

הקוד בודק אם המשימה כבר בסטטוס בוצע. אם כן — הוא מחזיר הודעה "המשימה נרשמה כהושלמה" בלי לזרוק שגיאה. זה מונע באגים מלחיצה כפולה.

ש: איך עובד StaffAllocator?

`StaffAllocator` קורא ל-`GET /api/repairs/available-workers/:category`. השרת מחזיר עובדות עם התמחות מתאימה + מספר משימות פתוחות (עומס). המזכירה רואה מי פנויה ושובצת בהתאם.

ש: מה ההבדל בין RepairWorkerList ל-ProductionStation?

(`/repairs/tasks`) `RepairWorkerList` — מציג רק משימות תיקון. `ProductionStation` (`/production`) — מציג גם פאות חדשות וגם תיקונים דרך `getWorkerUnifiedTasks` — תצוגה מאוחדת לעובדת. שני המסכים משתמשים ב-`TaskItem` המשותף.

ש: מהם סטטוסי התיקון?

בתיקון — משימות ייצור/תיקון פעילות. בחפיפה — הפאה בשלב שטיפה ועיצוב. בבקרה — ממתינה לאישור QA. מוכן — אושרה, ממתינה למסירה. נמסר — נמסרה ללקוחה (עם התראת WhatsApp).

ש: אילו בדיקות כתבת?

`repair.test.ts` — יצירת תיקון, שליפת משימות לעובדת, סימון "בוצע", בדיקה שהמערכת מעבירה אוטומטית לשלב הבא. הבדיקות רצות ב-CI עם Jest.